

NED with open LLMs and domain ontologies

This chapter covers

- Understanding the limitations of traditional NED tools
- Combining general-purpose LLMs and domain ontologies for NED
- Performing multistep disambiguation with shortest-paths detection, path-to-text translation, and textual paths summarization

Chapter 7 is focused on named entity disambiguation (NED), highlighting the role of scispaCy, a specialized natural language processing (NLP) tool built on the spaCy framework. This tool is designed to process documents and publications by providing pretrained models in the biomedical domain.

8.1 Understanding limitations of traditional NED systems

scispaCy incorporates vocabularies and ontologies, such as the Unified Medical Language System (UMLS), that provide canonical entities useful for disambiguating mentions in the text. However, this approach has some limitations:

- It is designed for a particular application domain: the biomedical field.
- It presents challenges in expanding and updating the reference knowledge base to incorporate new entities and terms.
- It fails to fully use the extensive information available in the knowledge base.
- It doesn't use the existing relationships and paths between entities for the disambiguation task.

To understand the effect of this last point, let's recap the example we discussed at the beginning of chapter 7, which used this quote from the European Centre for Disease Prevention and Control (ECDC):

In the week of 13 April, Belize reported for the first time mosquito-borne Zika virus transmission. Update on the observed increase of congenital Zika syndrome and other neurological complications Microcephaly and other fetal malformations potentially associated with Zika virus infection.

scispaCy uses the contextual words surrounding the term “Zika” to detect the three correct disambiguated entities. The results are shown in the following listing.

Listing 8.1 Candidate selection and ranking with scispaCy

Recognized entity: Zika virus 75 85	Target entity “Zika Virus” detected between characters 75 and 85
Ranked target candidates:	
- C0318793 Zika Virus	←
- C0276289 Zika Virus Infection	
- C4687930 Zika Virus Antibody Measurement	
Recognized entity: congenital Zika syndrome 135 159	Target entity “Congenital Zika Syndrome” detected between characters 135 and 159
Ranked target candidates:	
- C4546023 Congenital Zika Syndrome	←
Recognized entity: Zika virus infection 268 288	Target entity “Zika Virus Infection” detected between characters 268 and 288
Ranked target candidates:	
- C0276289 Zika Virus Infection	←
- C0318793 Zika Virus	
- C4687930 Zika Virus Antibody Measurement	

But now let's test the following slightly different example that doesn't include surrounding words such as “congenital” and “syndrome”:

Zika belongs to the Flaviviridae virus family, and it is spread by Aedes mosquitoes. Individuals affected by Zika disease and other syndromes like chikungunya fever often experience symptoms like viral myalgia, infectious edema, and infective conjunctivitis. Severe outcomes of Zika are due to its capacity to cross the placental barrier during pregnancy, causing microcephaly and congenital malformations.

Compared to the previous example, there are no words supporting the disambiguation phase. Let's look at the scispaCy output.

Listing 8.2 Candidate selection and ranking with the scispaCy model

```

Recognized entity: Zika 0 4
Ranked target candidates:
- C0276289 Zika Virus Infection
- C0318793 Zika Virus
- C4687930 Zika Virus Antibody Measurement

Recognized entity: Zika disease 109 121
Ranked target candidates:

Recognized entity: Zika 278 282
Ranked target candidates:
- C0276289 Zika Virus Infection
- C0318793 Zika Virus
- C4687930 Zika Virus Antibody Measurement

```

The model disambiguates “Zika” as the entity “C0276289 Zika Virus Infection” in the first and the third sentences, but it doesn’t detect a target entity for the mention in the second sentence.

This chapter addresses these limitations with a novel approach that uses open large language models (LLMs) and domain ontologies. Unlike domain-based tools such as scispaCy, this approach can be used in other application domains in which we can use rich ontologies.

8.2 *Ingesting the domain ontology*

To drive the disambiguation process, we will use the SNOMED (Systematized Nomenclature of Medicine) ontology introduced in chapter 7. As a reminder, SNOMED is a multilingual clinical terminology repository encompassing more than 450,000 concepts and a rich set of relationship types. For our example scenario, we will again use these two files: `sct2_Description_Full-en_US1000124_20220901.txt` (entity names and aliases, and relationships between entities) and `sct2_Relationship_Full_US1000124_20220901.txt` (numerical codes identifying entities and relationships).

NOTE For examples from these files, see section 7.5.2.

Figure 8.1 illustrates the SNOMED hierarchical structure and the propagation of information through the nodes. Listings 8.3–8.5 do the following, respectively: create nodes and relationships in Neo4j from `sct2_Relationship_Full_US1000124_20220901.txt`, extract names and aliases from `sct2_Description_Full-en_US1000124_20220901.txt`, and follow the hierarchical structure to propagate information from first-level nodes to deeper nodes.

NOTE For annotated versions of these listings and more details, see section 7.6.4. The complete example code is available in the book’s online repository.

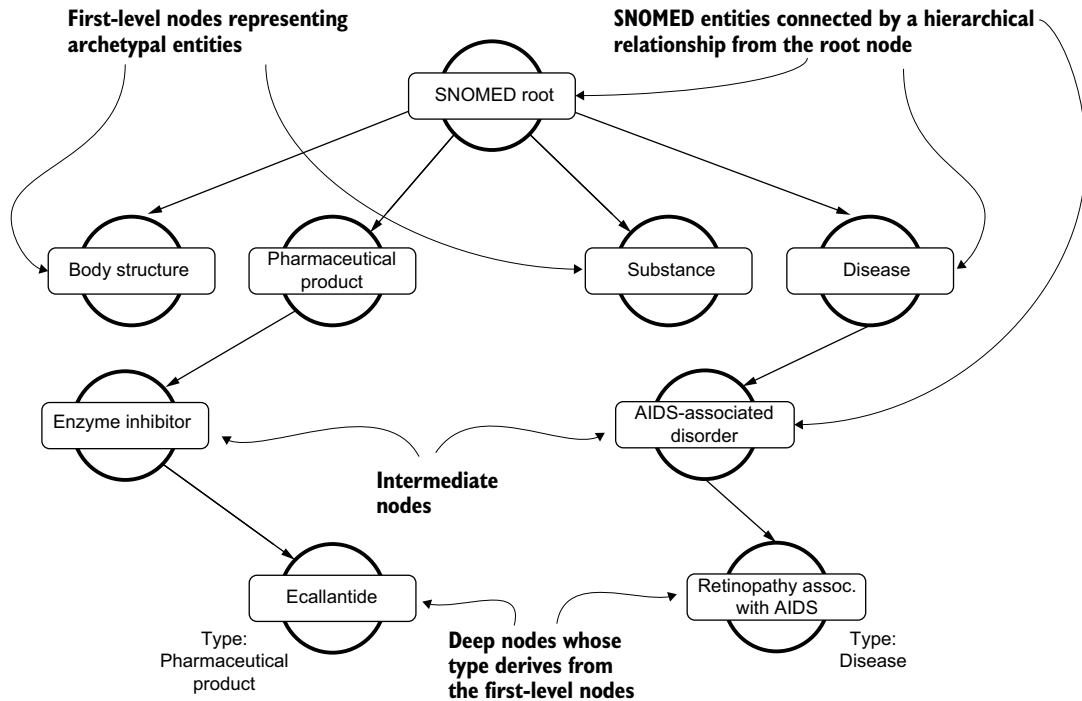


Figure 8.1 A sample of SNOMED's hierarchical structure. Nodes on deeper levels can be categorized using information from first-level nodes, which are the ontology's archetypal entities.

Listing 8.3 Ingesting SNOMED: loading relationships

```
[...]
class SnomedRelationshipsImporter(BaseImporter):
    [...]
    def set_constraints(self):
        queries = [
            (
                "CREATE CONSTRAINT IF NOT EXISTS FOR (n:SnomedEntity) "
                "REQUIRE n.id IS UNIQUE"
            ),
            (
                "CREATE INDEX snomedNodeName IF NOT EXISTS "
                "FOR (n:SnomedEntity) ON (n.name)"
            ),
            (
                "CREATE INDEX snomedRelationId IF NOT EXISTS "
                "FOR ()-[r:SNOMED_RELATION]-() ON (r.id)"
            ),
            (
                "CREATE INDEX snomedRelationType IF NOT EXISTS "
                "FOR ()-[r:SNOMED_RELATION]-() ON (r.type)"
            ),
        ]
```

```

        "CREATE INDEX snomedRelationUmls IF NOT EXISTS "
        "FOR () - [r:SNOMED_RELATION] - () ON (r.umlsls)"
    ),
]

for q in queries:
    self.connection.query(q, db=self.db)

def import_snomed_rels(self):
    query = """
        UNWIND $batch as item
        MERGE (e1:SnomedEntity {id: item.sourceId})
        MERGE (e2:SnomedEntity {id: item.destinationId})
        MERGE (e1)-[:SNOMED_RELATION {id: item.typeId}]->(e2)
        FOREACH(ignoreMe IN CASE WHEN item.typeId = '116680003'
        ➡ THEN [true] ELSE [] END|
            MERGE (e1)-[:SNOMED_IS_A]->(e2)
        )
    """

    size = self.get_csv_size(snomedRels_file)
    self.batch_store(snomed_rels_query, self.get_rows(snomedRels_file),
                    size=size)

```

Listing 8.4 Ingesting SNOMED: loading names and aliases

```

[...]
```

```

class SnomedNamesImporter(BaseImporter):
    [...]

    def import_snomed_names(self, snomedNames_file):
        snomed_names_concepts_query = """
            UNWIND $batch as item
            MATCH (e1:SnomedEntity)
            -[r:SNOMED_RELATION {id: item.conceptId}]->
            (e2:SnomedEntity)
            WHERE item.conceptId <> '116680003' AND r.id = item.conceptId
            SET r.type = CASE
                WHEN r.type IS NULL THEN item.termAsType
                ELSE r.type END,
            r.aliases = CASE
                WHEN item.termAsType IN r.aliases THEN r.aliases
                ELSE coalesce(r.aliases, []) + item.termAsType END
        """

        snomed_names_entities_query = """
            UNWIND $batch as item
            MATCH (e:SnomedEntity {id: item.conceptId})
            SET e.name = CASE
                WHEN e.name IS NULL THEN item.term
                ELSE e.name END,
            e.aliases = CASE

```

```

        WHEN item.term in e.aliases THEN e.aliases
        ELSE coalesce(e.aliases, []) + item.term END
    """
    size = self.get_csv_size(snomedNames_file)
    self.batch_store(
        snomed_names_concepts_query,
        self.get_rows(snomedNames_file),
        size=size)
    self.batch_store(
        snomed_names_entities_query,
        self.get_rows(snomedNames_file),
        size=size)
[...]
```

Listing 8.5 Ingesting SNOMED: labeling propagation from first-level nodes

```

[...]
```

```

class SnomedLabelPropagator():
[...]
```

```

    def get_rows(self):
        propagation_query = """
            MATCH p=(n:SnomedEntity)<-[:SNOMED_IS_A]-(m:SnomedEntity)
            WHERE n.id= "138875005" // Root node
            WITH distinct m as first_node

            CALL apoc.path.expandConfig(first_node, { // #A
                relationshipFilter: '<SNOMED_IS_A',
                minLevel: 1,
                maxLevel: -1,
                uniqueness: 'RELATIONSHIP_GLOBAL'
            }) yield path

            UNWIND nodes(path) as other_level // #B
            WITH first_node, collect(DISTINCT other_level) as uniques
            UNWIND uniques as unique_other_level
            WITH first_node, unique_other_level
            WHERE not first_node.name in
            coalesce(unique_other_level.type, [])

            RETURN unique_other_level.id as id, first_node.name as label - C
        """

        with self._driver.session(database=self._database) as session:
            result = session.run(query=propagation_query)
            for record in iter(result):
                yield dict(record)
[...]
```

The `SNOMED_IS_A` relationship has been used to propagate the semantic types through the tree structure by using the hierarchical connections between entities.

8.3 Setting up the model with Ollama and Llama 3.1 8B

In previous chapters, we explored how to use OpenAI APIs to perform NLP tasks. Now, we extend that knowledge by deploying a NED system locally using Ollama and Llama 3.1 8B (released by Meta).

Ollama is an open source tool that lets users run LLMs directly on their local machines. By running models locally, we gain full control over our data while reducing latency and dependency on external providers. Llama 3.1 8B is an open source LLM with 8 billion parameters. It supports a context length of up to 128,000 tokens and is optimized for multilingual information processing. It is also designed for efficient deployment on consumer-grade hardware.

To deploy the Llama 3.1 8B model on your local machine, you will first need to download and install Ollama. The tool is compatible with macOS, Linux, and Windows, and the installation files are available at <https://ollama.com/>. Ollama offers both command-line and graphical user interface (GUI) options; we used the following command-line instructions to download and deploy Llama 3.1 8B for our NED system.

Listing 8.6 Ollama commands to download and install Llama 3.1 8B

```
ollama serve
ollama pull llama3.1:latest
```

Ollama offers built-in compatibility with the OpenAI Chat Completions API, so we can interact directly with our locally deployed model using Python code like that in previous chapters. This simplifies the process of integrating the model into our NED system. The next listing shows the Python class needed to interact with our model.

Listing 8.7 Running our Llama 3.1 8B model in Python

```
from openai import OpenAI

class LLM_Model():
    def __init__(self, url='http://localhost:11434/v1', key="default"):
        self.client = OpenAI(
            base_url= url,
            api_key = key,
        )
    def generate(self, messages):
        response = self.client.chat.completions.create(
            model="llama3.1:latest",
            messages=messages,
            temperature=0,
            max_tokens=4000,
            top_p=1,
            frequency_penalty=0,
            presence_penalty=0,
        )
        # It assumes as response the ChatGPT API format
        return response.choices[0].message.content
```

← Default URL through which the model is served

← "api_key" is required for the OpenAI Chat Completions API but is not used for open models.

← Specifies llama3.1:latest, downloaded by Ollama

NOTE We obtained our results for the example, generated using the prompts in the following sections, in October 2024 using the latest version of the Llama 3.1 model.

Using a general-purpose model like Llama 3.1 for NED will demonstrate the potential of LLMs to perform well in niche areas when combined with domain-specific ontologies. In the next section, we break down this process and show how the system can interpret and disambiguate entities in complex biomedical texts, facilitating information extraction and analysis.

8.4 End-to-end NED process

Figure 8.2 shows the mental model describing our example’s NED process from the input document to the disambiguated mentions. The process begins with an input document containing unstructured text, which is analyzed by an LLM-based named entity recognition (NER) component. Here, the LLM identifies and labels relevant biomedical entities in the input text using the knowledge embedded in the domain ontology. For instance, a term like “Zika” can be recognized as a *Disease* concept according to SNOMED. This step transforms raw text into structured data, which will be processed in subsequent stages.

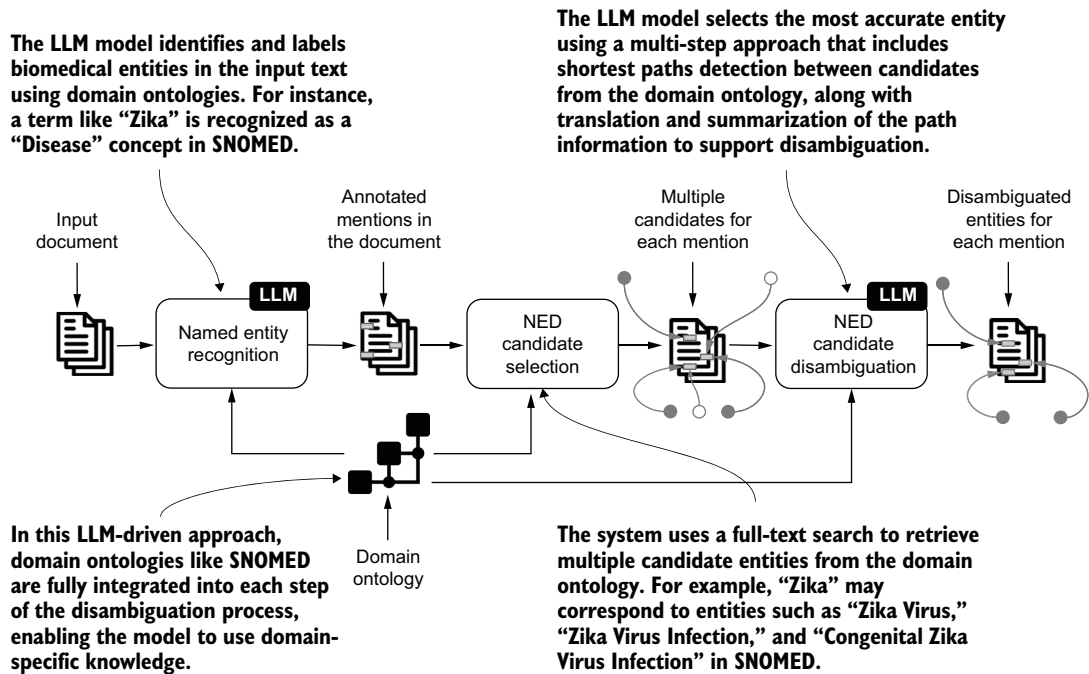


Figure 8.2 Workflow for a NED system designed to use LLMs and domain-specific ontologies, such as SNOMED, for biomedical text processing. Each stage of this workflow involves interactions to accurately disambiguate entities in the input text.

Next, the system moves to the NED candidate selection (CS) stage. A full-text search mechanism generates a list of possible matches for each identified entity mention. For instance, the term “Zika” may correspond to several entities in SNOMED, such as *Zika Virus*, *Zika Virus Infection*, and *Congenital Zika Virus Infection*. This step sets up a pool of potential disambiguation targets; the system will evaluate them in the following phase to find the most accurate match.

In the final stage, NED candidate disambiguation (CD), the LLM refines its selection to determine the most precise entity match for each mention. A multistep approach identifies the shortest path between candidates in the ontology structure; it then translates and summarizes relevant path details to support disambiguation with contextual knowledge and ensure that each entity mentioned in the input document is mapped accurately to its corresponding disambiguated entity in the ontology.

This LLM-driven approach integrates domain-specific ontologies at each step of the disambiguation process. By incorporating the hierarchical and relational structures of the ontology, the model can make more informed decisions about entity classification and disambiguation, especially in complex cases where terms may have multiple meanings or associations.

8.4.1 *Named entity recognition*

The goal of NER is to identify and classify named entities mentioned in unstructured text into predefined categories, such as diseases, organisms, and procedures. As discussed in previous chapters, one practical approach is to use prompt engineering, where the types of entities we are interested in are explicitly defined in the prompt. Often, a data scientist or data engineer works with a domain expert to identify and define these entities.

In our scenario, we incorporate SNOMED’s structured medical knowledge to enable the LLM to perform more precise and contextually aware entity recognition in biomedical texts. Figure 8.3 illustrates the input and output of the NER.

NER requires us to retrieve all the predefined categories from the ontology. The following query retrieves the categories from SNOMED.

Listing 8.8 Retrieving predefined categories from SNOMED

```
MATCH (n:SnomedEntity)
UNWIND n.type as named_entity
WITH DISTINCT named_entity, count(named_entity) as num_of_entities
ORDER BY num_of_entities DESC
RETURN collect(named_entity) as named_entities
```

We can use the results from this query in our prompt for the NER task. Listing 8.9 is a simplified version of the prompt messages we defined for this purpose; the full prompt is in code repository.

The LLM identifies and labels biomedical entities in the input text using domain ontologies. For instance, a term like “Zika” is recognized as a “Disease” concept in SNOMED.

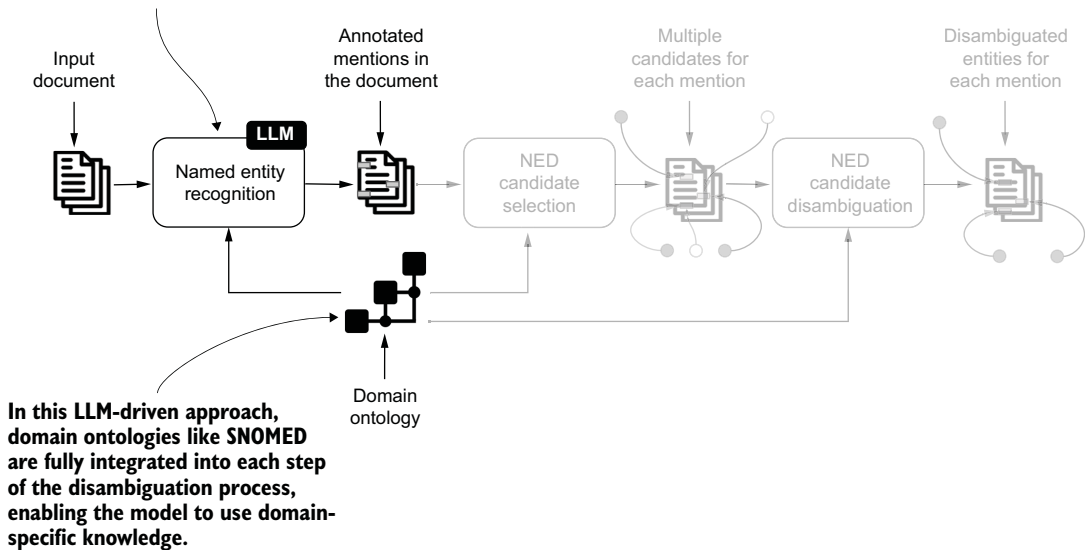


Figure 8.3 The first stage of NED is NER. In our scenario, named entities are derived directly from the ontology. In SNOMED, categories are defined by the ontology’s first-level nodes, whose information is propagated to all the other nodes.

Listing 8.9 Simplified prompt for NER

```
system = """You are an assistant capable of extracting named entities in the
➡medical domain.
Your task is to extract ALL single mentions of named entities from the text.
You must only use one of the pre-defined entities from the following list:
{named_entities}.
No other entity categories are allowed.
For each sentence, extract the named entities and present the output in
valid JSON format""".format(named_entities=named_entities))

input = """Risk factors for rhinocerebral mucormycosis include poorly
controlled diabetes mellitus and severe immunosuppression."""

assistant = [
    {
        "sentence": """Risk factors for rhinocerebral mucormycosis include
poorly controlled diabetes mellitus and severe immunosuppression.""",
        "entities": [
            {
                "id": 0,
                "mention": "Risk factors",
```

```

        "label": "Events"
    },
    {
        "id": 1,
        "mention": "rhinocerebral mucormycosis",
        "label": "Disease"
    },
    {
        "id": 2,
        "mention": "poorly controlled diabetes mellitus",
        "label": "Disease"
    },
    {
        "id": 3,
        "mention": "severe immunosuppression",
        "label": "Qualifier value"
    }
  ]
}
]

```

This prompt is structured as follows:

- *System instruction*—The system message says to extract occurrences of named entities that belong to categories in the medical domain from the SNOMED ontology, avoiding irrelevant or out-of-scope categories.
- *Input text*—In this example, the input text discusses risk factors for a medical condition (rhinocerebral mucormycosis), listing conditions such as diabetes mellitus and immunosuppression that are medical terms which need to be recognized.
- *Assistant output*—The system responds with structured JSON output that contains the following fields:
 - *sentence*—The system processes text sentence by sentence to ensure that it analyzes each unit of meaning individually.
 - *entities*—The output includes an array of identified entities. Each entry contains an ID to uniquely identify the entity in the sentence, a mention of the named entity found in the text (e.g., “rhinocerebral mucormycosis”), and a label that classifies the entity according to a SNOMED category, such as *Disease*.

Here’s an example of input text passed to the LLM instructed with the NER prompt.

Listing 8.10 User message for NER

```

user = ""Severe outcomes of Zika are due to its capacity to cross the
placental barrier during pregnancy, causing microcephaly
and congenital malformations"".

```

And here is a subset of the results generated by Llama 3.1.

Listing 8.11 NER output from the sentence in listing 8.10

```
{
  "sentence": "\"Severe outcomes of Zika are due to its capacity to cross
the placental barrier during pregnancy, causing microcephaly
and congenital malformations.\"",
  "entities": [
    {
      "id": 0,
      "mention": "Zika",
      "label": "Organism",
      "start": 19,
      "end": 22
    },
    {
      "id": 1,
      "mention": "microcephaly",
      "label": "Clinical finding (finding)",
      "start": 105,
      "end": 116
    },
    {
      "id": 2,
      "mention": "congenital malformations",
      "label": "Clinical finding (finding)",
      "start": 122,
      "end": 145
    }
  ]
}
```

From this result, we can identify and classify three distinct entities:

- “Zika”—Identified as an *Organism*, with its position in the sentence spanning from character 19 to 22.
- “Microcephaly”—Labeled a *Clinical finding (finding)*, indicating that it is a medical condition. This mention appears from character 105 to 116.
- “Congenital malformations”—Also labeled a *Clinical finding (finding)*. It is located between characters 122 and 145 in the sentence.

LLMs have trouble accurately detecting the starting and ending characters of mentions in a sentence. For this reason, we generated the `start` and `end` fields in post-processing with the following Python function.

Listing 8.12 Python function to compute starting and ending characters of a mention

```
def find_all_mention_indices(self, string, substring):
    indices = []
    start_index = 0
```

```

while True:
    start_index = string.find(substring, start_index)

    if start_index == -1:
        break # No more occurrences found

    end_index = start_index + len(substring) - 1
    indices.append((start_index, end_index))

    # Move start_index forward to search for the next occurrence
    start_index += len(substring)

return indices

```

This function identifies the position of the mention, which is information recognized by traditional NER systems.

8.4.2 Candidate selection

The second phase of the NED process is CS, which identifies relevant entities or concepts that could match the intended meaning of each identified named entity. Figure 8.4 illustrates the input and output of this phase, highlighting how candidate entities are selected.

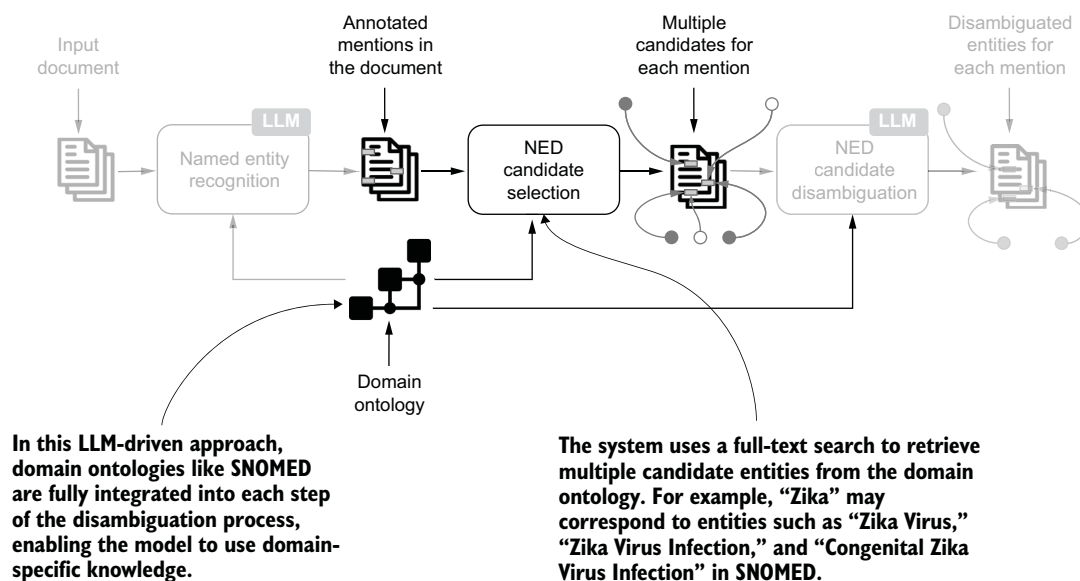


Figure 8.4 The second stage of NED is candidate selection. For each entity mention detected in the previous step, this stage retrieves potential candidates that may refer to it.

As shown in the following listing, the input for CS consists of the mentions annotated in the input text by the NER process, along with the domain ontology. The output is a list of one or more candidate entities associated with each mention.

We do not use the LLM in this step for two reasons. First, we want to retrieve candidates directly from the domain ontology rather than rely on knowledge embedded in the LLM. Second, the size of the ontology prevents us from loading it in its entirety in a prompt. So, to perform CS efficiently, we use Neo4j's full-text search capabilities, which can identify strings in the ontology that closely match each mention.

Listing 8.13 Python class for CS

```
class CandidateSelection:
[...]
```

```
    def full_text_query(self):
        query = """
            CALL db.index.fulltext.queryNodes("names", $fulltextQuery,
            ↳{limit: $limit})
            YIELD node
            WHERE node:SnomedEntity AND ANY(x IN node.type
            ↳WHERE x IN $labels)
            RETURN distinct node.name AS candidate_name, node.id
            ↳AS candidate_id
            """

        return query

    def generate_full_text_query(self, input):
        full_text_query = ""
        words = [el for el in input.split() if el]

        if len(words) > 1:
            for word in words[:-1]:
                full_text_query += f" {word}~0.80 AND "
                full_text_query += f" {words[-1]}~0.80"
            else:
                full_text_query = words[0] + "~0.80"

        return full_text_query.strip() [...]
```

We specify the `$labels` parameter to reduce the query's search space. These labels are collected from the output of the NER phase, forcing the system to identify only a subset of entities that are relevant to the type of entities mentioned.

NOTE Although the full-text search mechanism offers a viable solution, it can be enhanced by incorporating a vector-based search to retrieve additional candidates that may not be identified through text matching.

The JSON result of the query when we pass “Zika” as an input term is shown next.

Listing 8.14 Example of updated NED results from the CS step

```

{
  "id": 0,
  "mention": "Zika",
  "label": "Organism",
  "start": 19,
  "end": 22,
  "candidates": [
    {
      "snomed_id": "50471002",
      "name": "Zika virus"
    },
    {
      "snomed_id": "3928002",
      "name": "Zika virus disease"
    },
    {
      "snomed_id": "762725007",
      "name": "Congenital Zika virus infection"
    }
  ]
}

```

The `candidates` field includes a list of potential matches or candidates the system found for each mention. Each candidate represents a possible interpretation of the mention based on SNOMED. The following fields characterize each candidate:

- `snomed_id`—The unique identifier for the concept in SNOMED
- `name`—The name of the medical entity associated with the `snomed_id`

In this case, the candidates are “Zika virus” (50471002), “Zika virus disease” (3928002), and “Congenital Zika virus infection” (762725007). These candidates represent possible medical meanings of “Zika” in clinical terminology, setting the stage for further refinement in the disambiguation phase.

8.4.3 Candidate disambiguation

The final phase of the NED process is CD (see figure 8.5). We apply a strategy that uses contextual information provided by other medical entities co-occurring with the target entities in a sentence. By cross-referencing these entities with the structured knowledge in the domain ontology, we can verify and refine the selected candidates to determine the most accurate match.

For example, consider a sentence that mentions both “Zika” and “microcephaly.” The presence of “microcephaly” alongside “Zika” provides valuable context: it suggests an association with *Congenital Zika virus infection*, given that this infection is known to cause microcephaly. The disambiguation process can use this co-occurrence to prioritize *Congenital Zika virus infection* over other potential meanings of “Zika” (such as a general virus or an unrelated term).

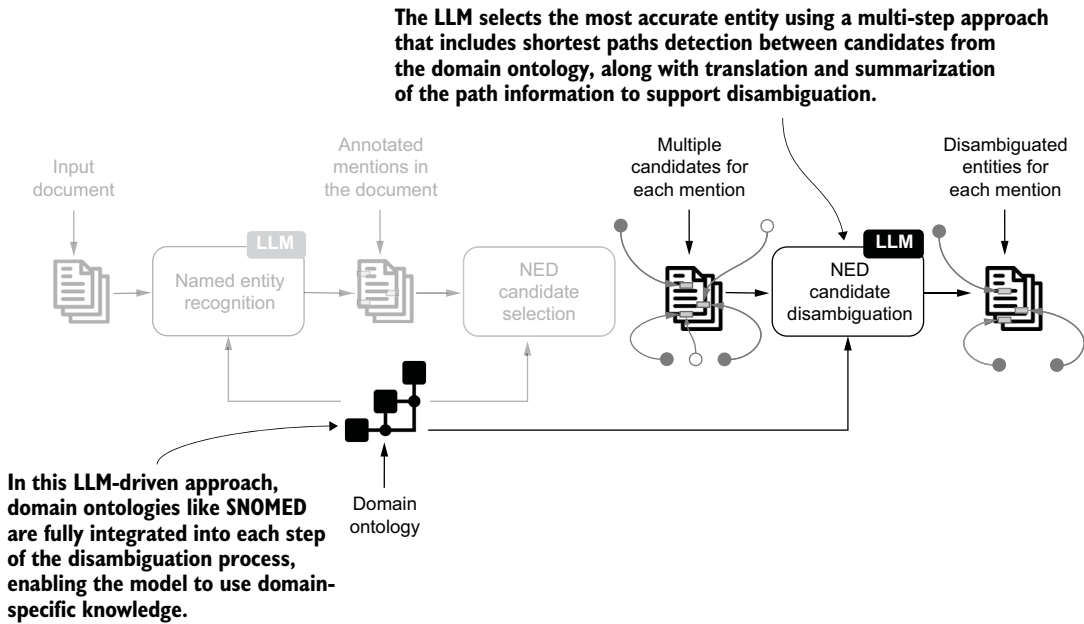


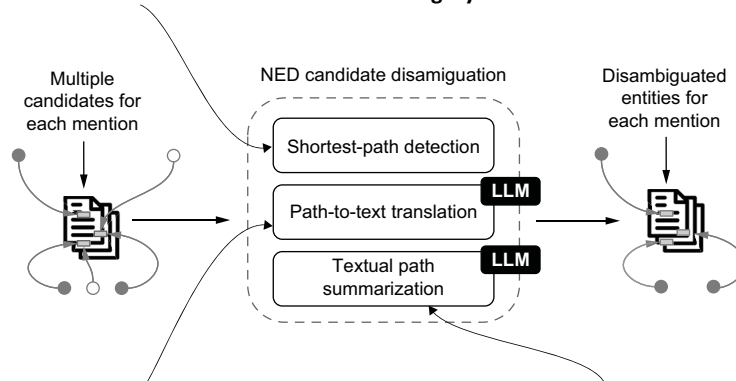
Figure 8.5 The third stage of NED is the candidate disambiguation. The goal is to select the best match among all the possible candidates identified in the previous step, by combining graph-based algorithms (shortest-path detection) and LLMs.

To generate disambiguated entities for each mention identified in the input document, we perform three steps:

- 1 *Detecting shortest paths*—We identify the minimal-length paths between the candidate entities associated with different mentions in a sentence. By mapping these connections, we establish potential relationships that help clarify the intended meaning of each mention.
- 2 *Translating paths to text*—To use the LLM’s strengths in processing textual information, we translate each graph path that connects candidate entities into natural language sentences. This transformation enables the LLM to interpret relational information in a format it processes effectively.
- 3 *Summarizing textual paths*—We summarize all textual information derived from the translated paths into a synthetic explanation. This summary captures the essence of the relationships and supports the LLM in making more accurate disambiguation decisions.

Figure 8.6 provides an overview of these steps in the disambiguation process. LLMs empower the path-to-text translation and textual path summarization steps, enabling them to interpret and condense relational information effectively. Shortest-path detection, discussed next, uses Neo4j’s Graph Data Science (GDS) library to identify connections between candidates.

Step 1. Detect the shortest paths between identified candidates in a sentence. For example, (Congenital Zika virus infection) - [:OCCURRENCE] -> (Congenital) <- [:OCCURRENCE] - (Micrencephaly) represents the path between “Zika virus disease” and “Chikungunya fever.”



Step 2. Translate the shortest paths between identified candidates into sentences. For example, “A Congenital Zika virus infection occurrence is associated with a Congenital occurrence, which in turn is associated with Micrencephaly.” is the textual translation of the previous path.

Step 3. Summarize all textual shortest paths into a unique paragraph that will be used for disambiguation.

Figure 8.6 NED CD steps: (1) detecting the shortest paths between all the candidates related to the entity mentions in the sentence; (2) translating detected paths into natural language sentences; (3) summarizing those sentences into useful text for disambiguation

DETECTING SHORTEST PATHS

The goal of this step is to detect the shortest path between all the possible candidates associated with each medical entity mention identified during the CS phase. The following listing shows the query to perform this operation.

Listing 8.15 Python class for extracting relevant path from the SNOMED ontology

```
class PathExtraction():
    def __init__(self, model, store, candidates, named_entities):
        self.model = model
        self.store = store
        self.candidates = candidates
        self.named_entities = named_entities
    [...]

    def get_co_occs_query(self, s1_id, s2_id):
        query = f"""
            CALL gds.degree.stream('snomedGraph')
            YIELD nodeId, score
            WITH gds.util.asNode(nodeId).name AS name, score AS degree
            ORDER BY degree DESC
            LIMIT 350
            WITH collect(name) as hub_nodes
            MATCH (s1), (s2)
```

```

WHERE s1.id="{s1_id}" AND
      s2.id="{s2_id}"
WITH s1,
      s2,
      allShortestPaths((s1)-[:SNOMED_RELATION*1..2]-(s2)) AS paths,
      hub_nodes
UNWIND paths AS path
WITH relationships(path) AS path_edges, nodes(path) as path_nodes,
      hub_nodes
WITH [n IN path_nodes | n.name] AS node_names,
      [r IN path_edges | r.type] AS rel_types,
      [n IN path_edges | startnode(n).name] AS rel_starts,
      hub_nodes
WHERE not any(x IN node_names WHERE x IN hub_nodes)
WITH [i in range(0, size(node_names)-1) | CASE
WHEN i = size(node_names)-1
THEN "(" + node_names[size(node_names)-1] + ")"
WHEN node_names[i] = rel_starts[i]
THEN "(" + node_names[i] + ")" + '-[:' + rel_types[i] + ']->'
ELSE "(" + node_names[i] + ")" + '<-[:' + rel_types[i] + ']-' END]
  ➡as string_paths
RETURN DISTINCT apoc.text.join(string_paths, '') AS `Extracted paths`
""".format(s1_id=s1_id, s2_id=s2_id, named_entities=named_entities)
return query
[...]
```

The critical steps of this query are as follows:

- *Degree calculation*—The query first retrieves nodes with the highest “degree” (number of relationships) from the graph using `CALL Gds.Degree.Stream`. These nodes represent highly connected hub nodes, which will later be excluded to focus on more meaningful, less generic connections.
- *Shortest-path search*—The query finds all shortest paths between two entities, `s1` and `s2`, based on their IDs, limiting the path length to one or two hops (relationships). The relationships between these nodes are filtered to exclude “hub” nodes and avoid generic or overly broad relationships.
- *Path transformation*—Once paths are identified, the query unwinds them and collects both the nodes and relationships involved in each path. It then formats these paths into readable strings that show the direction and types of relationships (e.g., `(n1) -[:REL_TYPE] -> (n2)`).

The next listing shows an example of detected paths.

Listing 8.16 Paths detected using the Neo4j GDS library

```
[
  {
    "id": 1,
    "path": "(Congenital Zika virus infection)-[:OCCURRENCE]->(Congenital)
  ➡<-[:OCCURRENCE]-(Micrencephaly)"
  },
  {
```

```

    "id": 2,
    "path": "(Congenital Zika virus infection)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Acrocephaly)""
  },
  {
    "id": 3,
    "path": "(Congenital Zika virus infection)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Multiple congenital malformations)"
  },
  {
    "id": 4,
    "path": "(Congenital Zika virus infection)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Congenital malformation)"
  },
  {
    "id": 5,
    "path": "(Congenital Zika virus infection)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-([X]Other congenital malformations)"
  },
  {
    "id": 6,
    "path": "(Micrencephaly)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Multiple congenital malformations)"
  },
  {
    "id": 7,
    "path": "(Micrencephaly)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Congenital malformation)"
  },
  {
    "id": 8,
    "path": "(Micrencephaly)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-([X]Other congenital malformations)"
  },
  {
    "id": 9,
    "path": "(Acrocephaly)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Multiple congenital malformations)"
  },
  {
    "id": 10,
    "path": "(Acrocephaly)-[:IS_A]->(Craniosynostosis syndrome)
    ➡<-[:IS_A]->(Congenital malformation)"
  },
  {
    "id": 11,
    "path": "(Acrocephaly)-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]
    ➡<->(Pathological developmental process)
    ➡<-[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]-(Congenital malformation)"
  },
  {
    "id": 12,
    "path": "(Acrocephaly)-[:OCCURRENCE]->(Congenital)
    ➡<-[:OCCURRENCE]-(Congenital malformation)"
  }

```

```

    },
    {
      "id": 13,
      "path": "(Acrocephaly)-[:OCCURRENCE]->(Congenital)
      ↳<-[:OCCURRENCE]-([X]Other congenital malformations)"
    }
  ]

```

In this JSON output, each entry contains an ID and a path, and each path shows a relational link between biomedical entities based on their co-occurrence in ontology paths. Here are the details:

- *Congenital Zika virus infection paths*—Many paths begin with *Congenital Zika virus infection* linked to various congenital conditions, such as *Micrencephaly*, *Acrocephaly*, and *Multiple congenital malformations*, through the relationship type `[:OCCURRENCE]`. This implies that *Congenital Zika virus infection* is associated with these conditions, possibly as a cause or occurrence.
- *Shared congenital condition*—*Congenital* is a common node linking multiple congenital conditions like *Micrencephaly* and *Acrocephaly*. This central node indicates that these conditions share a similar occurrence attribute.
- *Alternative relationships*—Some paths use relationships like `[:IS_A]` and `[:PATHOLOGICAL_PROCESS_(ATTRIBUTE)]`, showing hierarchical or attribute-based relationships. For instance, *Acrocephaly* is classified under *Craniosynostosis syndrome*, which is linked to *Congenital malformation*.

TRANSLATING PATHS TO TEXT

This step translates graph paths into sentences, which allows LLMs to process complex relational data in a format they are optimized to understand: natural language. This transformation makes it easier for the model to interpret connections between entities, providing context that aids in distinguishing between similar terms. The following is a simplified version of the prompt for translating paths into sentences.

Listing 8.17 Simplified version of the prompt for translating paths into sentences

```

system = """You are an assistant capable of translating a Neo4j graph path
↳into a clear sentence.
Use the exact entity names from the path while generating the sentence.
The sentences will assist a large language model (LLM) in disambiguating
↳biomedical entities.
Ensure the output is a valid JSON with no extraneous characters."""

input = {
  "path": "(Hypertension)-[:RISK_FACTOR_FOR]->(Cardiovascular Disease)
  ↳<-[:ASSOCIATED_WITH]- (Myocardial Infarction)"
}

assistant = {
  "sentence": "Hypertension is a risk factor for cardiovascular
  ↳disease. Myocardial infarction is also associated with cardiovascular

```

```

    ➡disease, indicating that hypertension may increase the risk of
    ➡experiencing a myocardial infarction through its connection to
    ➡cardiovascular disease."
  }

```

Here are the details of the prompt:

- *System instruction*—The system translates Neo4j graph paths into clear, human-readable sentences.
- *Input graph paths*—In this case, the input consists of a graph path from a Neo4j database, which might represent complex relationships.
- *Assistant output*—The system responds with a valid JSON structure showing the generated sentence.

Each graph path is translated into a single, clear sentence, as shown in the following listing.

Listing 8.18 Results of the path's translation into natural language

```

[ {
  "sentence": "A Congenital Zika virus infection occurrence is associated
  ➡with a Congenital occurrence, which in turn is associated with
  ➡Micrencephaly."
},
{
  "sentence": "A Congenital Zika virus infection occurrence is associated
  ➡with a Congenital occurrence, which in turn is associated with an
  ➡Acrocephaly occurrence."
},
[...],
{
  "sentence": "Micrencephaly occurs in Congenital and Multiple congenital
  ➡malformations also occur in Congenital."
},
{
  "sentence": "Micrencephaly occurs in Congenital and is also an occurrence
  ➡of Congenital malformation."
},
[...],
{
  "sentence": "Acrocephaly occurs in Congenital and Other congenital
  ➡malformations also occur in Congenital."
} ]

```

These JSON entries make the graph paths accessible for an LLM to interpret and use for disambiguation.

SUMMARIZING TEXTUAL PATHS

Before executing the final disambiguation of the selected candidates, we need to summarize the sentences representing the translated graph paths. Doing so reduces the “cognitive load” for the model (fewer tokens), making it easier to interpret entity

relationships and select the most accurate candidate without being overwhelmed by excessive detail. The next listing shows a simplified version of the prompt.

Listing 8.19 Simplified prompt for summarizing textual paths

```
system = ""You are an assistant that can summarize multiple sentences
↳ derived from ontology paths into a short summary. This summary will be
↳ used to support a named entity disambiguation task.
Ensure the output is a valid JSON with no extraneous characters.""

input = {
  "sentences": [
    {
      "id": 1,
      "sentence": "Hypertension is a risk factor for cardiovascular
↳ disease. Myocardial infarction is also associated with cardiovascular
↳ disease, indicating that hypertension may increase the risk of
↳ experiencing a myocardial infarction through its connection to
↳ cardiovascular disease."
    },
    {
      "id": 2,
      "sentence": "Diabetes mellitus is a complication that arises from
↳ an endocrine disorder. Diabetic retinopathy is also associated with
↳ endocrine disorders, suggesting that diabetes mellitus can lead to the
↳ development of diabetic retinopathy through its link to endocrine
↳ dysfunction."
    },
    {
      "id": 3,
      "sentence": "Asthma is associated with respiratory disorders.
↳ Allergic rhinitis is also linked to respiratory disorders, which
↳ implies that individuals with asthma may also experience allergic
↳ rhinitis due to their common association with respiratory
↳ conditions."
    },
    {
      "id": 4,
      "sentence": "Osteoporosis leads to bone weakness. Bone fractures
↳ are a result of bone weakness, indicating that osteoporosis can
↳ increase the likelihood of bone fractures due to the weakened state
↳ of the bones."
    }
  ]
}

assistant = {
  "context": "Hypertension is a risk factor for cardiovascular disease,
↳ which in turn increases the likelihood of experiencing a myocardial
↳ infarction. Similarly, diabetes mellitus is linked to endocrine disorders,
↳ potentially leading to complications such as diabetic retinopathy. Asthma
↳ and allergic rhinitis are both associated with respiratory disorders,
↳ suggesting a common link between these conditions. Finally, osteoporosis
↳ weakens bones, making individuals more susceptible to bone fractures."
}
```

Here are the details of the prompt:

- *System instruction*—The system is instructed to summarize sentences derived from ontology paths, retaining all identified entities in the summaries. The output should be formatted as a valid JSON object, with each summary provided as a string under the key `context`.
- *Input sentences*—The input consists of multiple sentences, each containing relationships between medical conditions and their effects or associations.
- *Assistant output*—The system responds with a single summarized sentence for each group of related entities in a valid JSON format.

The output structure summarizes the core relationships in the input sentences, preserving the critical entities and relationships (listing 8.20).

Listing 8.20 Results of the summarization phase

```
{
  "context": "A Congenital Zika virus infection occurrence is associated
  ➡with various congenital malformations, including Micrencephaly,
  ➡Acrocephaly, Multiple congenital malformations, and Other congenital
  ➡malformations. These conditions all share a common link to the Congenital
  ➡entity."
}
```

This result provides the LLM with a distilled version of complex relational information, allowing it to focus on the most relevant context for disambiguation.

DISAMBIGUATION

In the final stage, we combine all the ingredients for the disambiguation, including the selected candidates and the textual details provided by the summarization phase. The next listing shows the prompt for the disambiguation.

Listing 8.21 Prompt for final disambiguation

```
system = """You are an assistant specialized in entity disambiguation.
Your task is to identify and accurately disambiguate the entities
➡mentioned in a given sentence, relying heavily on the contextual entities
➡present in surrounding sentences:
1. Original Sentence: The sentence that contains ambiguous entities that
   need to be resolved.
2. Candidate Entities: A list of potential entities extracted from the
   sentence, with each entity having multiple possible meanings or labels.
3. Contextual Sentences: A collection of related or surrounding sentences
   that provide additional context for disambiguating the mentioned entities.
Your objective is to use the entities mentioned in the contextual sentences
➡as the primary source of information to disambiguate the entities in the
➡original sentence. Analyze the candidate entities for each ambiguous
➡mention and select the one that aligns best with both the context and the
➡meaning provided by the contextual sentences. The output must be a valid
➡JSON. """
```

```

input = {
  "sentence": "Asthma and allergic rhinitis are commonly addressed
  ➡together in treatment protocols, given their shared underlying
  ➡inflammatory processes in allergic individuals.",
  "candidates": [
    {
      "id": 1,
      "candidates": [
        {
          "snomed_id": "233681001",
          "name": "Extrinsic asthma with asthma attack"
        },
        {
          "snomed_id": "195967001",
          "name": "Asthma"
        },
        {
          "snomed_id": "266361008",
          "name": "Intrinsic asthma"
        },
        {
          "snomed_id": "266364000",
          "name": "Asthma attack"
        },
        {
          "snomed_id": "270442000",
          "name": "Asthma monitored"
        },
        {
          "snomed_id": "170642006",
          "name": "Asthma severity"
        },
        {
          "snomed_id": "170643001",
          "name": "Occasional asthma"
        },
        {
          "snomed_id": "170644007",
          "name": "Mild asthma"
        },
        {
          "snomed_id": "170645008",
          "name": "Moderate asthma"
        }
      ]
    }
  ],
  "context": "Asthma is associated with respiratory disorders. Allergic
  ➡rhinitis is also linked to respiratory disorders, which implies that
  ➡individuals with asthma may also experience allergic rhinitis due to their
  ➡common association with respiratory conditions."
}

```



```

assistant = {
  "entities": [
    {
      "id": 1,
      "disambiguation": {
        "snomed_id": "195967001",
        "name": "Asthma"
      }
    },
    {
      "id": 2,
      "disambiguation": {
        "snomed_id": "61582004",
        "name": "Allergic rhinitis"
      }
    }
  ]
}

```

Here are the details of the prompt:

- *System instruction*—The system is directed to identify and accurately disambiguate ambiguous entities. It must analyze each entity in the original sentence and select the candidate that best aligns with the contextual meaning provided by the summarized contextual information, prioritizing entities that align with these contextual details.
- *Input structure*:
 - *Original sentence*—The primary sentence contains ambiguous entities that require disambiguation.
 - *Candidate entities*—A list of potential SNOMED entities for each mention, each with multiple possible interpretations or labels.
 - *Contextual sentences*—Additional sentences that provide context to help clarify the meaning of each ambiguous entity in the original sentence.
- *Assistant output*:
 - *id*—A unique identifier for the entity mention
 - *disambiguation*—An object with the selected SNOMED entity, including *snomed_id* and *name*, that best matches the contextual information

We achieved the following result by passing the LLM the input sentence “Severe outcomes of Zika are due to its capacity to cross the placental barrier during pregnancy, causing microcephaly and congenital malformations.”

Listing 8.22 Results of the disambiguation process

```

{
  "entities": [
    {
      "id": 0,

```

```

    "disambiguation": {
      "snomed_id": "762725007",
      "name": "Congenital Zika virus infection"
    }
  },
  {
    "id": 1,
    "disambiguation": {
      "snomed_id": "204030002",
      "name": "Micrencephaly"
    }
  },
  {
    "id": 2,
    "disambiguation": {
      "snomed_id": "116022009",
      "name": "Multiple congenital malformations"
    }
  }
]
}

```

Each entity is matched to the most relevant concept in the SNOMED ontology, allowing for precise identification and classification based on contextual information.

8.5 Conclusions

This chapter provided an in-depth exploration of a NED system that uses open LLMs and domain-specific ontologies. By integrating domain ontologies like SNOMED with open and general-purpose LLMs such as Llama 3.1 8B, we can address some of the limitations of traditional NLP tools in the biomedical domain, such as scispaCy. Our flexible approach can adapt across application domains: using Neo4j's GDS library for shortest-path detection and full-text search, combined with the disambiguation power of LLMs, enables a robust system for identifying and accurately disambiguating entities in complex texts. Through techniques like path-to-text translation and textual path summarization, we improved the LLM's ability to process relational data in a natural language format, enhancing its capacity to distinguish between similar entities. This framework lays the groundwork for future applications of LLMs in domain-specific NED tasks.

Summary

- Named entity disambiguation is essential for accurately identifying and distinguishing entities in complex domains.
- Traditional NLP tools such as scispaCy can't be used in diverse domains and can't use the relationships between entities, and their reference knowledge can't be extended and updated.
- Combining general-purpose LLMs and domain ontologies allows us to address these problems: LLMs can be driven by the continuously updated knowledge incorporated by the ontology, and use its relational structure.

- We deployed a flexible end-to-end process for NED, including multiple phases involving LLMs and domain ontologies.
- To take full advantage of the capabilities of LLMs combined with the graph dimension of domain ontologies, disambiguation is divided into three stages: detecting shortest paths, translating paths to text, and summarizing text paths.
- Future NED applications can use this framework and adapt to other domains characterized by rich ontologies describing the relational nature of their entities.